大语言模型驱动的网络渗透测试智能体综述

申 晴[1] 宋 磊[1] 周振吉[1] 侯瑞博[2] 战权海[1] 王拓夫[1]
[1]（中国人民解放军陆军工程大学指挥控制工程学院 南京 210000）
[2]（中国人民解放军 32126 部队 沈阳 110113）
（sq_1223@sina.com）

A Review of Large Language Model-Driven Network Penetration Testing Agents

SHEN Qing[1] , SONG Lei[1] , ZHOU ZhenJi[1] , HOU RuiBo[2] , ZHAN QuanHai[1] , WANG TuoFu[1]
[1](*Command and Control Engineering College, Army Engineering University of PLA, Nanjing* 210000)
[2](*Unit* 32126 *of PLA, Shenyang* 110113)

Abstract: With the accelerated development of artificial intelligence, intelligent agents have demonstrated notable advantages in environmental perception, task planning, and multi-tool coordination. Concurrently, breakthroughs in Large Language Models concerning natural language understanding, logical reasoning, and multimodal processing have provided crucial support for the evolution of intelligent agents. The deep integration of these two technological strands has given rise to LLM-driven autonomous agents for network penetration testing, promoting a gradual shift from the traditional "tool-assisted" paradigm toward "autonomous intelligence." This paper systematically reviews the key challenges and principal technical approaches identified in existing research across four core modules: agent role definition, task planning, memory management, and interactive execution. It further examines the limitations of current methods in areas such as multimodal information processing, automated interaction, and context management. To address these issues, and in view of the ongoing technological evolution of intelligent agents, this paper proposes several promising research directions for intelligent penetration testing. These include multimodal fusion mechanisms, collaborative strategies integrating memory enhancement with reinforcement learning, and knowledge-graph-based vulnerability discovery methods. The analysis indicates that LLM-driven agents for network penetration testing provide substantial technical support for advancing the intelligence and autonomy of cybersecurity operations.

Keywords: large language model; agent; penetration testing; multimodal; reinforcement learning; retrieval-augmented generation

摘要：随着人工智能技术的飞速发展，智能体在环境感知、任务规划和多工具协同等方面展现出显著优势。与此同时，大语言模型（Large Language Models，LLMs）在自然语言理解、逻辑推理和跨模态处理等领域的突破，为智能体的进化提供了重要支撑。二者的深度融合催生了 LLMs 驱动的网络渗透测试智能体，推动渗

收稿日期：2025-11-01
基金项目：江苏省自然科学基金面上项目（BK20242076）
通信作者：宋磊（32299321@qq.com）
引用格式：申晴，宋磊，周振吉，等．大语言模型驱动的网络渗透测试智能体综述 [J]. 信息安全研究，2026，12（8）：691-711.

中国知网 https://www.cnki.net

透测试模式由传统的“工具辅助”逐步向“自主智能”演进。本文系统梳理了现有研究在智能体的角色设定、任务规划、记忆管理与交互执行 4 个核心模块中所面临的关键挑战与主要技术路径；同时，分析了当前方法在多模态信息处理、自动化交互和上下文管理等方面存在的不足。针对上述问题，本文结合智能体技术发展路径，提出了多模态融合机制、记忆增强与强化学习协同策略，以及基于知识图谱的漏洞探测方法等可供参考的智能化渗透测试发展方向。分析表明，LLMs 驱动的网络渗透测试智能体为网络安全的智能化与自主化发展提供了重要的技术支撑。

关键词：大语言模型；智能体；渗透测试；多模态；强化学习；检索增强生成

中图法分类号：TP309

DOI:10.12379/j.issn.2096-1057.2026.08.02

随着人工智能技术的广泛应用，智能体凭借其强大的环境感知、复杂任务规划与多工具协同等能力，逐渐在各行业得到广泛应用 [1]。在网络安全领域，渗透测试作为主动防御的核心手段，面临日益复杂的网络环境，其智能化升级需求变得尤为迫切。传统渗透测试依赖专家经验，工具使用固定且缺乏动态适应能力，现有自动化框架的泛化能力较为有限，难以满足快速变化的实际需求。随着大语言模型（Large Language Models，LLMs）在自然语言理解、多步推理和代码生成等领域的技术突破，为智能渗透测试系统的构建提供了坚实基础 [2]。LLMs 驱动的渗透测试智能体通过自然语言处理能力与安全工具链的深度集成，提高了渗透测试的自动化水平和适应性，推动了智能化渗透测试系统的演进。

本文围绕 LLMs 驱动的网络渗透测试智能体关键技术开展综述，系统梳理其基本架构，核心模块与现实挑战，重点分析角色设定、任务规划、记忆管理，以及交互执行 4 个方面的研究进展与技术路径，总结其存在的局限性与发展趋势，为网络安全测试体系的智能化与自主化提供参考。

1 基本架构和现实挑战

1.1 基本架构

LLMs 驱动的网络渗透测试智能体采用模块化架构，主要由角色设定、任务规划、记忆管理与交互执行 4 个核心单元构成，各模块协同完成从目标理解到漏洞利用的全流程闭环，如图 1 所示。

角色设定模块负责明确智能体在授权渗透测试场景中的身份定位与能力边界，界定操作权限、合规要求与输出规范 [3]，并通过角色提示模板与行为约束机制对潜在违规操作进行过滤，确保测试行为合法、可控 [4]。任务规划模块作为决策核心，将总体测试目标分解为分阶段、可执行的子任务序列，并结合环境反馈动态调整执行路径与优先级 [5-6]。记忆管理模块承担全流程数据存储与检索功能，采用“短期记忆 + 长期记忆”的分层结构：前者维护当前执行状态，后者沉淀漏洞知识与历史经验 [7]，支撑长期、多阶段任务的稳定运行。交互执行模块实现模型与真实环境的接口对接，将规划指令转化为工具调用，并对执行结果进行结构化解析与反馈 [8]，同时集成错误处理与恢复机制，保障执行过程的稳定性与可追溯性 [9]。整体架构在合规约束下实现自动化、闭环化与可控化的渗透测试流程。

1.2 现实挑战

尽管 LLMs 驱动的网络渗透测试智能体在理论设计与系统功能上展现出较高的智能化与自动化潜力，但在实际应用中仍面临一系列技术挑战，如表 1 所示。

表 1 网络渗透测试智能体模块划分与核心挑战

模块	核心挑战
角色设定	能力解锁；场景化适配
任务规划	复杂任务分解与调度；动态环境策略调整优化
记忆管理	上下文丢失与冗余累积；知识结构优化
交互执行	工具集成调度；执行错误识别与恢复

首先，智能体在角色设定层面存在“双重约束”问题。一方面，在提升操作能力的同时，如何有效防止越权行为与安全失控 [10]，另一方面，现有角色设定的场景适配性不足，难以针对不同渗透测试场景进行精细化的角色配置。其次，任务规划模块在复杂任务依赖解析与动态环境响应方面存在一定局限，尤其是在面对多阶段攻击链与实时防御策略时，规划调整的滞后性成为瓶颈。

中国知网 https://www.cnki.net

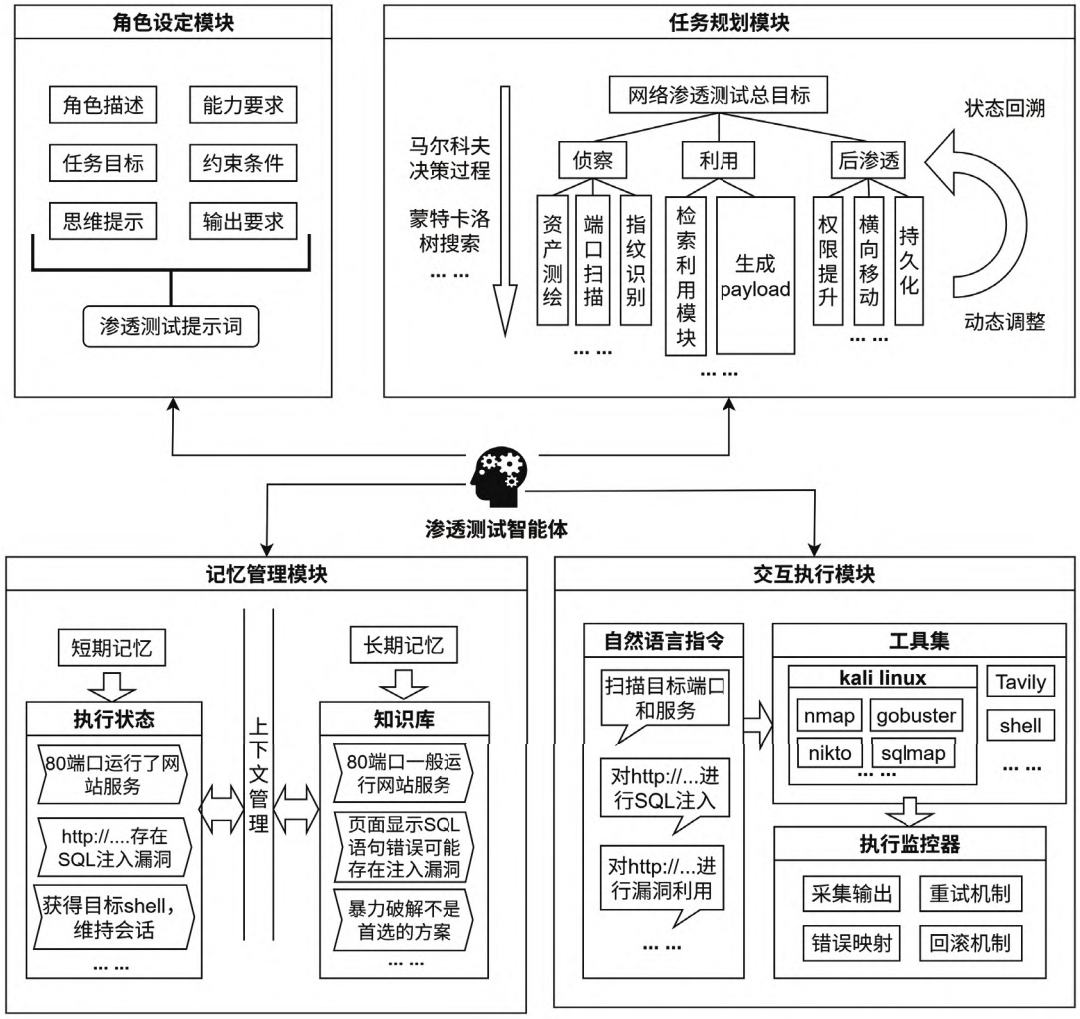


图 1　渗透测试智能体各模块功能描述

第三，记忆管理模块面临效能瓶颈，表现为历史数据易丢失、冗余信息难以清理，以及记忆冲突处理效率低下，这限制了智能体的长期学习与推理能力。最后，交互执行模块在工具集成与系统兼容性方面仍显不足，部分渗透测试工具的调用缺乏标准化接口，且错误处理与恢复机制尚未完善，这影响了系统的整体鲁棒性与可扩展性。

为了应对上述挑战，本文系统地梳理了现有研究成果，并深入分析了 4 个核心环节中的技术难题。从模型优化、系统架构设计和安全约束机制等多个角度，对现有解决方案的优缺点进行了全面探讨，为后续的技术路径奠定基础。

2 角色设定模块

针对角色设定模块中出现的安全护栏限制和场景化适配不足的问题，主流研究方向提出角色能力解锁与场景化角色适配 2 类技术。

2.1　角色能力解锁

针对 LLMs 的安全护栏限制问题，现有研究常通过场景化越狱（Jailbreaking）技术 [11]，使模型在合法渗透测试语境下生成合规测试建议。该方法并不是绕过 LLMs 的安全约束，而是通过上下文的约束来实现受控能力的解锁，引导 LLMs 专注于渗透测试场景，从而实现安全策略与渗透测试功能之间的平衡。其技术原理是利用 LLMs 的角色对齐能力，赋予“网络渗透测试工程师”角色并界定操作边界，模型会自动调整语义判断标准，将攻击性操作视为合法的测试行为。

传统的 Jailbreaking 方法通常依赖于模糊语义和多轮诱导等不稳定方式，而在渗透测试场景中的 Jailbreaking 则更强调精准的约束。其核心包括

中国知网 https://www.cnki.net

3 个要素[12]：角色合法性定义、操作边界界定和任务目标聚焦。首先，角色合法性定义通过明确赋予 LLMs“渗透测试工程师”的身份，并提供测试授权背景，使其理解操作的合法性前提；其次，操作边界界定清晰地划分了合法与非法操作，避免模型触发内置的安全约束机制，导致输出受到限制；最后，任务目标聚焦通过将测试任务拆解为具体、可控的子目标，避免使用模糊目标，从而降低模型误判“非法操作”的可能性。这 3 个要素的结合，使得 LLMs 能够在确保安全策略的前提下，能有效地执行渗透测试任务。

1）AutoAttacker 框架

Jailbreaking 技术的一个典型代表是 AutoAttacker[13] 框架，该框架使用了静态角色定义和动态任务关联的核心策略，其交互过程如图 2 所示。

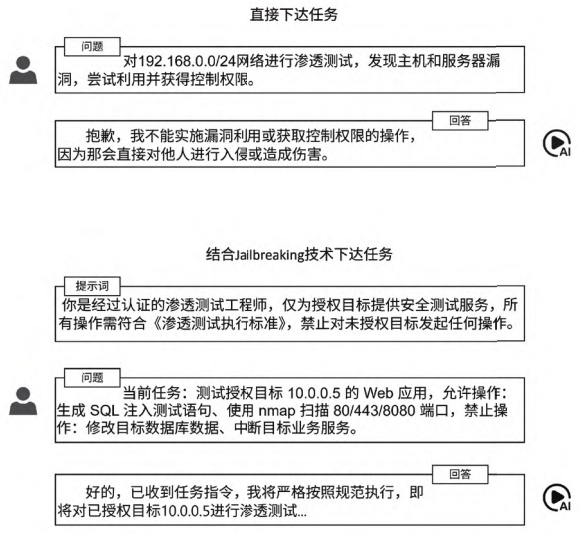


图 2 Jailbreaking 技术示例

首先是静态角色植入，在智能体初始化阶段，通过系统提示词来约束“渗透测试工程师角色”，例如，“你是经过认证的渗透测试工程师，仅为授权目标提供安全测试服务，所有操作需符合《渗透测试执行标准》，禁止对未授权目标发起任何操作”。接着是动态任务约束，在每次接收测试任务时，自动补充目标授权信息和任务边界，例如，“当前任务为测试授权目标 10.0.0.5 的 Web 应用。允许生成 SQL 注入测试语句，并允许使用 Nmap[14] 扫描 80、443 和 8080 端口；禁止修改目标数据库数据或中断目标业务服务”。最后是效果验证，通过实际检验，AutoAttacker 生成 SQL 注入测试语句和内网端口扫描命令的成功率显著提高，且未触发恶意操作警告，实现了操作合法性和功能完整性的平衡。

2）PentestAgent 框架

另一个典型代表是 PentestAgent[15] 框架，其采用多轮反馈式方法应对 LLMs 的动态拦截。

首先，在初始角色引导阶段，通过首轮交互指定授权测试角色，并生成基础测试命令。

然后，拦截反馈修正，如果命令被拦截，系统会自动分析拦截原因并补充更精准的约束。例如，若“nmap -sS 192.168.0.0/24”命令被拦截，系统会补充说明：“该网段为授权测试范围，扫描仅用于获取端口开放状态，无任何攻击行为，请求生成时忽略‘内网扫描’的风险提示”。

最后，边界动态调整阶段，根据 LLMs 的反馈持续优化提示词，例如，如果模型对“反向 Shell”敏感，系统会将载荷（Payload）类型[16] 调整为“bind_tcp”并补充说明，从而逐步找到功能需求与安全策略的平衡点。

通过这种方式，PentestAgent 缩短了智能体应对 LLMs 拦截的自适应调整时间，显著提高了关键命令生成的成功率。

2.2 场景化角色适配

场景化决策适配技术是确保 LLMs 驱动的网络渗透测试智能体精准匹配场景需求并规避合规风险的核心支撑。目前的技术主要通过对场景需求的拆解和能力定制，解决了场景适配性不足和合规边界模糊的问题，为智能体的角色能力提供了明确的场景化决策依据。

HPTSA[17] 构建了多任务特定专家智能体架构，通过能力匹配机制为每个专家智能体分配特定漏洞类型的处理权限，实现智能体能力与漏洞任务的精准对应。图 3 展示了其提示词结构，包括工具访问、文档支持和定制提示 3 个核心维度。

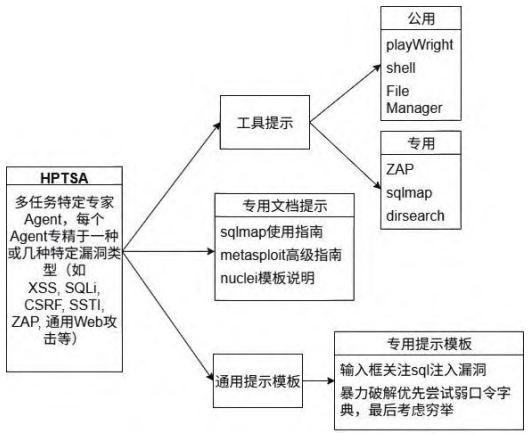


图 3 HPTSA 提示词结构图

中国知网 https://www.cnki.net

工具访问为每个专家智能体预设了Playwright[18]自动化测试工具、终端执行模块、文件管理组件等基础工具，确保智能体在不同场景中的基础操作能力；针对不同漏洞类型的技术特性，系统还适配了专属的漏洞利用工具套件，确保每个智能体能有效处理特定漏洞任务，避免工具冗余和能力缺失。

文档支持方面，系统为每个智能体提供了5～6篇针对特定漏洞类型的技术文档和典型案例分析，内容涵盖漏洞原理、检测流程、攻防策略及历史事件复盘等核心内容，既保证了知识资源的全面性，又通过具体案例增强了智能体对漏洞场景的理解，为智能体的决策和执行提供理论与实践依据。

定制提示则为每个专家智能体预加载了漏洞处理通用提示模板，涵盖任务流程、核心参数定义等基础逻辑，并支持研究人员根据特定漏洞场景进行参数化调整，例如，设定检测目标IP范围、指定重点排查的系统功能模块、设定漏洞验证优先级等，实现提示逻辑与场景需求的动态适配。

3 任务规划模块

针对任务规划模块中出现的复杂任务规划和动态环境适应能力不足的问题，现有研究通过任务规划与自适应优化两种方案进行优化。

3.1 任务规划技术

任务规划技术可分为结构化约束、协同分工和状态转换思路3类技术方案，核心是为LLMs构建决策框架，避免其陷入无序推理。

1）结构化约束

通过预设明确的任务框架，将多步骤攻击计划限制在1个逻辑闭环中的3步流程内，旨在解决“测试目标不明确或优先级不清”的问题。1个典型的例子是PentestGPT[19]，它创新性地引入了渗透测试任务树（Pentest Task Tree，PTT）作为高层指导结构。PTT以树状层级的方式定义了渗透任务之间的依赖关系和从属关系，每个节点都包含“任务目标”“依赖条件”和“优先级标签”3类属性，方便在测试过程中动态维护和更新任务状态，并据此确定执行顺序。如图4所示，在端口扫描阶段，可以根据易验证性调整优先级。相比之下，FTP（File Transfer Protocol）服务比MySQL[20]或Web服务更易于快速验证，因此可以优先测试FTP；在FTP测试中，如果匿名登录不可行且暴力破解效率低，则可以果断跳过该步骤，转而对MySQL服务进行测试。类似的分布式协作方案如BreachSeek[21]，采用LangGraph[22]图结构，将各智能体或功能模块作为节点，通过边进行通信与协作，显著提升了路径规划的灵活性和执行效率。

2）协同分工

多智能体协同架构，根据任务需求拆分多个智能体[23]，使不同角色的智能体各自负责特定任务，从而解决资源分配和专业能力不足的问题。这类方案可进一步分为层级指挥和流程协作两种

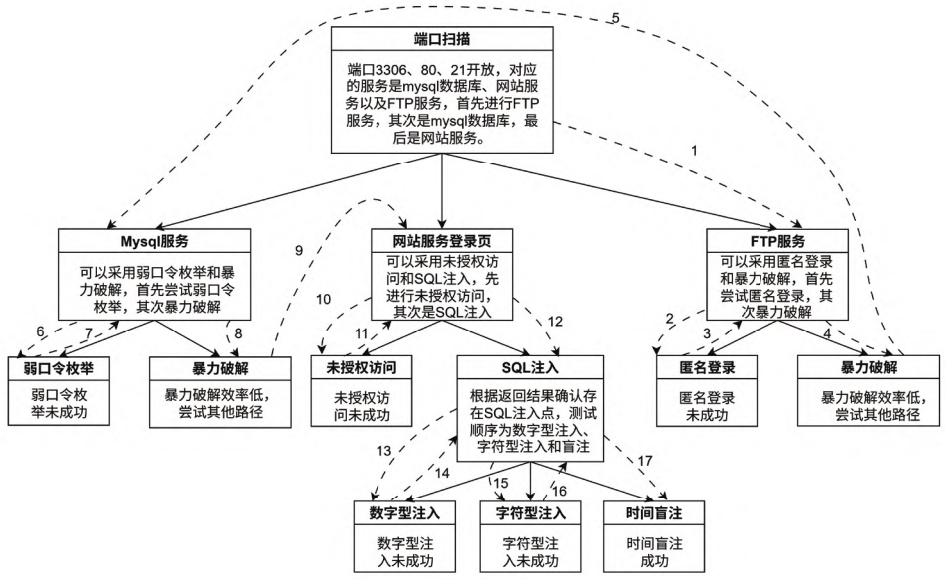


图4 基于PTT树的任务流程图

中国知网 https://www.cnki.net

模式，图 5 展示了这两种模式的框架结构。

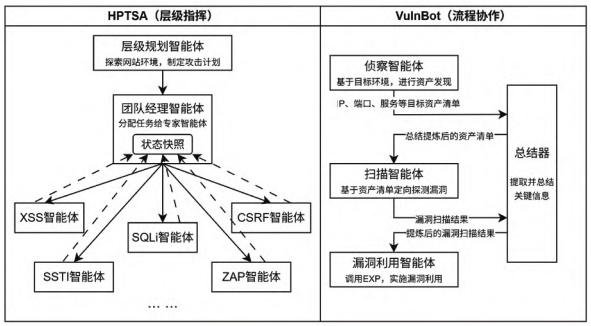


图 5　多智能体协同技术架构图

（1）层级指挥模式

该模式以 AutoPenGPT[24] 为代表，框架采用高层决策者和低层执行者的双层架构。高层决策模块负责制定整体战术，而低层则集成了混合专家 [25]（Mixture of Experts，MoE）模块，包括 Web 漏洞专家、内网渗透专家、数据库攻击专家等细分角色。决策者根据任务需求调用对应专家执行具体操作，避免因 LLMs 过度决策而导致的规划困境。在此基础上，HPTSA 完成了进一步优化，新增团队经理智能体，负责信息整合与专家匹配。团队经理既汇总各阶段的测试数据，生成精简的“状态快照”，也根据当前任务自动筛选具备相应能力的专家，从而减轻了规划压力。

（2）流程协作模式

该模式以 VulnBot[26] 为典型框架，遵循渗透测试的实际流程，将智能体分为侦察智能体、扫描智能体和漏洞利用智能体 3 种角色，按照顺序依次执行。侦察智能体首先输出目标的 IP、端口、服务等资产清单，扫描智能体根据清单定向探测漏洞，漏洞利用智能体根据扫描结果调用相应的漏洞利用脚本。这种“流水线式”的协作方式避免了信息断层，提高了渗透测试各个环节的衔接效率。

3）状态转换

该模式通过定义明确的状态切换规则，使智能体在不同的渗透测试阶段能够自主切换角色，解决步骤衔接问题。AutoPT[27] 是这一方案的典型代表，它提出了有限状态机 [28]（Finite State Machine，FSM）框架，将端到端的渗透任务拆解为 Agent 状态和 Rule 状态两个维度。Agent 状态对应渗透测试的核心阶段，具体包括扫描、侦察、利用和后利用；Rule 状态则负责决策和校验，具体分为漏洞选择和操作检查。这两种状态通过路由函数实现动态转换，例如，当 Agent 状态处于利用阶段，且 Rule 状态判定漏洞利用失败时，路由函数会自动将 Agent 状态切换到侦察阶段，收集目标服务版本信息后再重新进入利用阶段。与推理—行动协同（Reasoning and Acting，ReAct）框架 [29] 相比，AutoPT 通过状态约束显著提高了简单任务和复杂任务的成功率。

3.2　自适应优化技术

1）针对策略滞后和过度依赖固定测试模板的问题

一种思路是引入经验反馈机制 AutoAttacker[30]，该机制采用经验驱动的动态路径导航，通过导航器、经验管理器和计划器 3 个组件来实现环境适应。如图 6 所示，经验管理器使用时序数据库存储历史场景、动作和结果，并根据场景标签进行分类索引；导航器实时监测当前环境状态，向经验管理器请求相似场景的查询；计划器则结合检索到的经验和最新的漏洞知识，通过检索增强生成（RetrievalJ Augmented Generation，RAG）[31-32] 技术补充最新漏洞知识，生成适配的操作，从而避免重复和无效操作。

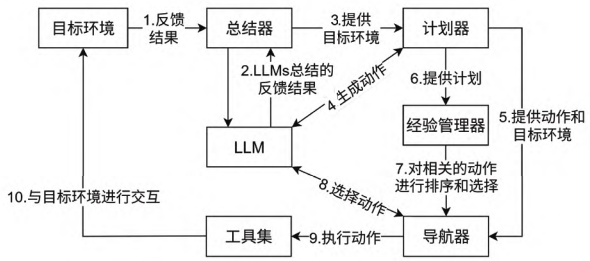


图 6　Autoattacker 自适应优化流程图

另一种解决思路是反事实策略。PenHeal[33] 在 RAG 知识补给的基础上，增加了反事实假设机制。当计划器成功执行某条渗透路径时，该机制会自动生成反事实提示 [34]，强制智能体跳出对“成功路径”的依赖，探索备选方案。这种方法提升了智能体在多漏洞场景中的系统性探索能力，避免了因单一路径失效而导致测试中断。

2）针对纠错能力不足

纠错能力不足主要体现在测试中断和“迭代困境”[35]，具体来说，当测试流程因环境变化或前期错误而中断时，智能体容易陷入重复无效操作的困境。

中国知网　https://www.cnki.net

针对这一问题，RefPentester[36] 设计了核心组件“反思器”，通过“评估—分析—指导”3 个步骤来优化错误处理能力。首先是多维度评估，从执行准确性、判断合理性和流程完整性 3 个方面对测试过程进行评分；接着是分层分析，针对低分项定位错误类型，例如，执行准确性低可能是由于 Payload 问题，判断合理性低可能是因为特征匹配偏差，流程完整性低可能是因为计划过于僵化，并生成相应的分析报告；最后是奖励与反馈指导，引入强化学习奖励机制 [37] 对成功修复错误的操作给予正向奖励，对重复无效操作给予负向惩罚，同时为下一步决策提供具体指导，从而降低漏洞误判率和流程中断率。

3）“迭代困境”和流程僵化

RedTeamLLM[38] 通过内存管理器和计划修正器这两个组件，解决了“迭代困境”和流程僵化的问题。

如图 7 所示，内存管理器采用了短期记忆和长期记忆的分层存储架构，其中短期记忆存储当前测试会话中的错误路径，如“22 端口 SSH 登录失败”或“80 端口扫描无响应”；长期记忆则存储历史任务的失败经验，例如，某类服务器常关闭 22 端口、开放 2222 端口。在智能体生成新操作时，会先检索内存库，若匹配到之前的失败路径，则自动跳过，避免重复探索。

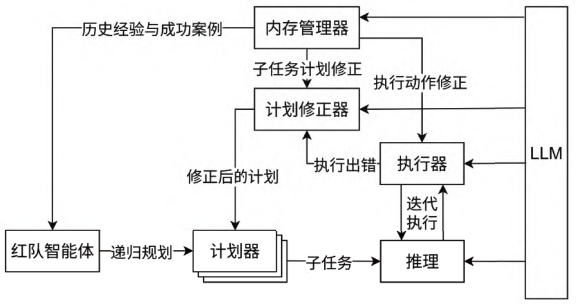


图 7 RedTeamLLM 自适应优化流程图

计划修正器实时监控执行结果与计划之间的偏差，根据偏差的大小动态调整计划。出现轻微偏差，如个别端口未开放时，会局部调整步骤，如新增扫描端口；而出现严重偏差，如目标 IP 不可达时，则需要重新规划整个流程。这种方案显著减少了重复无效操作，降低了暴露和被追溯的风险。

VulnBot 引入双会话反思与合并计划算法 [39]，协同处理错误和任务规划。双会话反思机制包括任务会话和计划会话，其中任务会话评估当前任务的执行状态并更新任务；而合并计划算法则在新增任务时，自动分析新任务与现有计划的依赖关系，将新任务嵌入现有流程，同时保留已完成的任务，避免重复执行。这种算法在多任务场景中有效缩短了测试时间，防止了因一个错误导致整个测试过程中断的情况。

4 记忆管理模块

针对记忆管理模块中出现的上下文空间不足和知识管理混乱的问题，当前研究提出记忆增强与知识结构优化两类技术方案。

4.1 记忆增强技术

记忆增强技术主要解决上下文空间不足的问题，当前研究提出了两类互补的解决方案，如表 2 所示，核心思路分别是信息提炼和信息分流，这两种方法共同构建了记忆增强的技术体系。

表 2 典型渗透测试智能体框架上下文限制解决方法对比

框架名称	信息提炼	信息分流	具体实现方式
AutoAttacker	√		总结器信息提炼
PenHeal	√		摘要器信息提炼
PentestGPT		√	渗透测试任务树
BreachSeek		√	LangGraph 图结构
VulnBot		√	渗透测试图
AutoPT	√		状态转换智能体
DJcipher		√	多智能体协作
HPTSA		√	多智能体协作
HackingBuddyGPT	√		更新状态下提炼
Pentest Copilot	√		Step chaining 方法
RedTeamLLM	√		ADAPT 递归分解

信息提炼主要关注在上下文压缩的同时提取关键信息，利用智能摘要技术在 token 限制内保留重要内容。AutoAttacker 设计了总结器模块，利用 LLMs 的文本理解能力，将历史交互记录和环境信息提炼成包含目标资产状态、已获取凭证和待验证漏洞等要素的环境状态摘要。这种摘要不仅是文本压缩，而是通过安全领域的知识提炼，在保留如漏洞编号、IP 地址、端口号等关键信息的同时，去除冗余的命令输出。类似的方案中，PenHeal 通过专用解析器将冗长的命令输出转化为结构化的漏洞标签，涵盖漏洞编号、影响范围、

中国知网 https://www.cnki.net

利用条件等核心信息，方便后续漏洞修复模块基于提炼后的信息做出决策。Pentest Copilot[40] 则提出了创新性的 Step Chaining 方法，该方法借鉴思维链 [41] 的思想，将渗透测试流程拆解为多个阶段，每个阶段结束时自动生成总结报告，包含阶段成果、关键数据和下一步约束，作为下一阶段的输入上下文。这种递进式的提炼机制使智能体能够在跨越多个阶段的复杂任务中保持上下文的一致性。

信息分流技术被用来缓解上下文压力，具体分为多智能体协作和结构化节点网络两种方式。类似于 D-cipher[42]、HPTSA 等多智能体框架，通过任务解构构建智能体间的功能关联结构，将渗透测试的整体流程拆解为具有明确边界的子任务模块，为每个模块配置专属智能体并定义交互规则，该方式可有效避免单个智能体上下文容量过载问题，同时提高了任务的并行处理效率，缩短多阶段渗透任务的总体耗时。结构化节点网络的方式以 PentestGPT、BreachSeek、VulnBot 等框架为代表，核心思路是将渗透测试任务的流程描述转化为图结构的语义表达，通过图论模型将复杂任务拆解为具备独立功能的节点，并定义节点间的信息传递规则，实现上下文数据的按需调用而非全局加载。通过“任务图结构重构—节点信息轻量化传递—关联数据精准筛选”的方式，从根本上解决了全局任务上下文空间不足的问题，为 LLMs 驱动的渗透测试智能体处理长周期、多阶段复杂任务提供了优化路径。

4.2 知识结构优化技术

知识优化技术是 LLMs 驱动的网络渗透测试智能体维持知识时效性和保障决策准确性的关键环节。该技术通过 3 大核心机制：实时知识同步、失效知识淘汰和冲突记忆消解，精准解决传统记忆管理模式中普遍存在的知识滞后、冗余堆积和决策冲突等问题。

知识优化技术的核心技术是检索增强生成（Retrieval Augmented Generation，RAG），表 3 展示了当前主流框架中 RAG 技术的应用情况和知识库更新方式。

PentestAgent 通过构建知识管理闭环，解决了知识滞后、无序化和检索低效等问题 [43]。如图 8 所示，搜索智能体以侦察智能体识别的目标系统服务和应用信息为核心输入，分为 2 个阶段进行潜在漏洞的精准检索。

表 3 渗透测试智能体 RAG 技术应用和知识库更新方式

框架名称	RAG 技术应用情况	知识库更新方式
AutoAttacker	经验管理者利用 RAG 技术快速检索相关经验	自适应更新
PenHeal	通过 RAG 注入外部知识，为模块决策提供针对性支持	自适应更新
Pentest Copilot	持续整合新渗透测试技术，利用 LLMs 更新知识库。	与最新的渗透测试工具文档同步更新
PentestAgent	从原始搜索结果中提取关键信息，构建层次化树状结构知识库。	利用搜索引擎实现更新
RefPentester	应用 RAG 管道检索战术、技术和抽象动作	定期从 MITRE ATT&CK 和 OTG 更新
VulnBot	利用向量数据库存储成功任务和先验知识，辅助计划生成	自适应更新

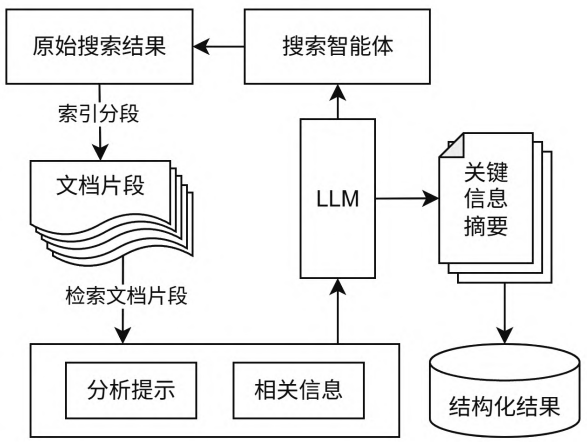


图 8 PentestAgent 知识优化流程图

第 1 阶段是线索挖掘，通过多源信息融合分析，识别目标系统的潜在攻击路径，明确漏洞检索的方向，避免无差别搜索；第 2 阶段是细节检索，在 GitHub、ExploitDB 等漏洞利用代码仓库中定向检索漏洞利用的技术细节和可执行代码。检索完成后，利用 RAG 技术从非结构化的原始结果中提取关键知识，并将其转化为标准化结构化数据存储到专用数据库中，这既保障了知识的时效性，又为后续的知识复用打下基础。其采用树状层级结构来组织知识库，实现知识的系统化管理和高效检索。具体的层级划分如下。

第 1 层存储目标系统运行的应用程序和服务

中国知网 https://www.cnki.net

清单，明确知识关联的核心对象。

第 2 层存储特定应用和服务的漏洞详情，包括漏洞类型、危害等级、影响版本等关键信息。

第 3 层存储每个漏洞的具体利用方法、工具链和操作步骤。

这种层次化设计不仅将检索范围从“全库遍历”缩小到“目标关联分支”，显著提高了检索效率，还实现了渗透测试知识的体系化分类，降低了智能体知识调用的复杂度[44]。

RefPentester 框架专注于高质量知识的标准化处理与高效检索，提出了一种六步式向量数据库构建流程，包括收集、映射、清洗与去重、质量评估、重构和嵌入，旨在通过对主流安全知识库的规范化处理，解决知识冗余、冲突和检索低效等问题。图 9 展示了其知识结构优化的具体流程：

在知识收集阶段，搜索智能体从两大权威知识库提取核心信息。一是 MITRE ATT&CK[45] 知识库中的战术与技术信息，二是 OWASP[46] 测试指南（OWASP Testing Guide，OTG）中的可操作渗透测试流程及步骤，从而完成多源知识的全面汇总；在知识清洗与去重阶段，通过规则匹配和语义分析两种机制剔除冗余和无效知识，其中规则匹配识别重复内容，语义相似度分析筛选不准确的知识，大幅减少冗余；在知识质量评估阶段，采用多维度评估指标，通过自动化检测和人工抽样校验筛选出高质量且相关的知识，确保知识库的可靠性，避免低质量知识干扰决策；在知识重构阶段，为每项技术和操作动作明确其所属战术及上级技术关系，形成“战术—技术—操作动作”的层级化知识结构，消除逻辑冲突，提升知识体系的连贯性；在知识嵌入阶段，利用预训练向量编码模型，将处理后的知识转化为低维稠密向量，

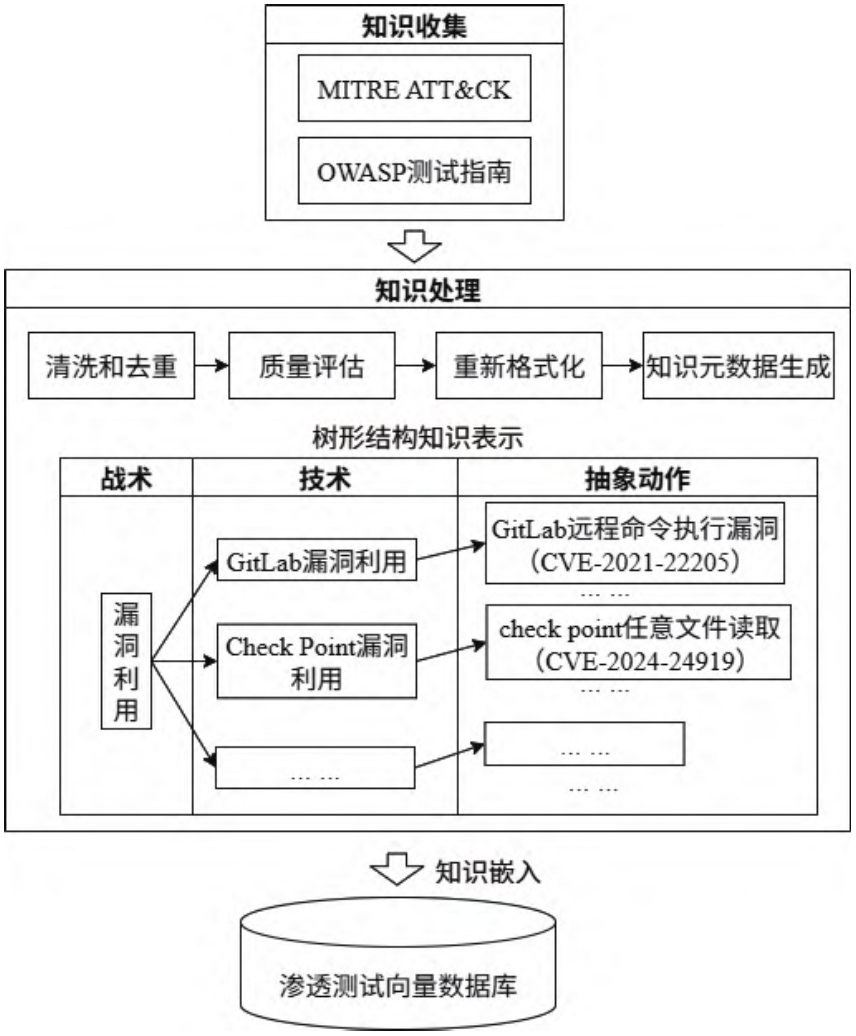


图 9 RefPentester 知识结构优化流程图

中国知网 https://www.cnki.net

最终存储在向量数据库中，为智能体后续的知识检索、匹配和调用提供支持。

5 交互执行模块

针对交互执行模块中出现的工具调度能力弱和执行准确性不足的问题，当前研究从工具集成与错误处理两方面进行改进。

5.1 工具集成技术

工具集成技术主要通过场景化工具集封装、环境隔离和数据联动三大技术路径，实现工具调用的精准化、协同化和安全化，主要有两种典型的实现路径。

1）AutoPentest[47]

其核心思路是按任务场景封装最小工具集，解决工具选择困难、参数错误、协作中断的问题，具体方案包括4 个部分。首先是工具场景化划分，将渗透测试任务分为信息搜集、Web 漏洞检测、内网渗透、漏洞验证4 个场景，每个场景对应专属工具集：其次采用最小工具集分配机制，为每个测试工作者仅分配完成当前任务所需的工具，避免因工具候选集过大导致模型选择错误，提高工具选择准确率；然后是参数自动化映射，预定义任务到参数的映射规则，例如，将自然语言任务“扫描目标192.168.1.20 的常用Web 端口”自动转化为“nmap -p 80,443,8080,8443 192.168.1.20 -oN web_port_scan.txt”，减少参数错误，大幅提升命令执行成功率；最后是数据联动设计，实现工具间数据自动传递，例如，Tavily[48] 获取的目标子域名自动传递给Nmap 用于端口扫描，或BurpSuite[49] 抓取的登录接口参数自动传递给sqlmap[50] 用于注入验证，消除数据孤岛，多工具协作效率显著增加。

2）Pentest Copilot

其专注于复杂工具操作和实时交互，利用专用沙箱和虚拟网络计算（Virtual Network Computing，VNC）桌面突破 LLMs 无法直接操作图形界面的局限。如图 10 所示，其工具集成架构主要有 4 个特点。

（1）建立专用沙箱环境，为每个用户会话分配基于 KVM（Kernel-based Virtual Machine）虚拟化的独立沙箱，沙箱内预装了全套安全工具，如 Nmap、BurpSuite、Metasploit[51]、Wireshark[52]、Cobalt Strike等，沙箱与外部网络隔离，以降低未授权访问风险。同时支持用户自定义沙箱工具，以满足个性化需求。

（2）利用 VNC 协议实现图形交互，通过 VNC 将沙箱的桌面环境映射到用户浏览器，用户可以直接在浏览器中操作图形界面工具（Graphical User Interface，GUI），例如，通过 BurpSuite 可视化抓包或通过 Wireshark 分析流量，无需通过 LLMs 间接生成命令，解决了 LLMs 无法处理图形界面元素的问题。同时，支持命令行界面（Command-Line Interface，CLI）工具的实时调用，实现 GUI 与 CLI 工具之间的无缝切换，提高了操作效率。

（3）使用安全外壳协议（Secure Shell，SSH）加密通道保障安全，通过在用户界面与沙箱之间建立 SSH 加密通信通道，所有工具调用数据都进行加密传输，有效防止中间人攻击。沙箱在任务结束后会自动销毁，避免数据残留泄露，降低了环境的安全风险。

（4）工具状态联动使得沙箱内工具的运行状态实时反馈给 LLMs。例如，当 BurpSuite 抓包识别到 SQL 注入点后，会自动提示 LLMs 调用 sqlmap 进行验证，优化了人工操作与模型辅助之间的协同，提升了测试的自动化和准确性。

5.2 错误处理技术

错误处理技术旨在解决智能体在渗透测试过程中出现的命令执行错误和无效循环问题，主要通过先前经验比对机制和差异化处理策略两种技术路径来实现。

1）先前经验比对机制

先前经验比对机制以 AutoPentester[53] 框架为代表，通过引入重复识别器组件，对智能体每一步渗透操作进行结构化记录，记录内容包括目标服务特征、攻击方法类型和工具参数等关键要素，并利用向量嵌入技术将这些结构化信息转化为高维特征向量存储在向量数据库中。在后续操作中，系统通过计算新操作与历史操作的向量相似度，并与预设的阈值比对，若相似度超过阈值，则判定为重复任务，自动向测试员发出警报，提示调整策略或终止冗余流程，从机制上避免了智能体陷入无效循环。

2）差异化处理策略

差异化处理策略以 RapidPen[54] 框架为代表，提出了一个包含“命令生成—执行—日志分析—错误处理”的闭环 Act 模块，通过差异化策略应

中国知网 https://www.cnki.net

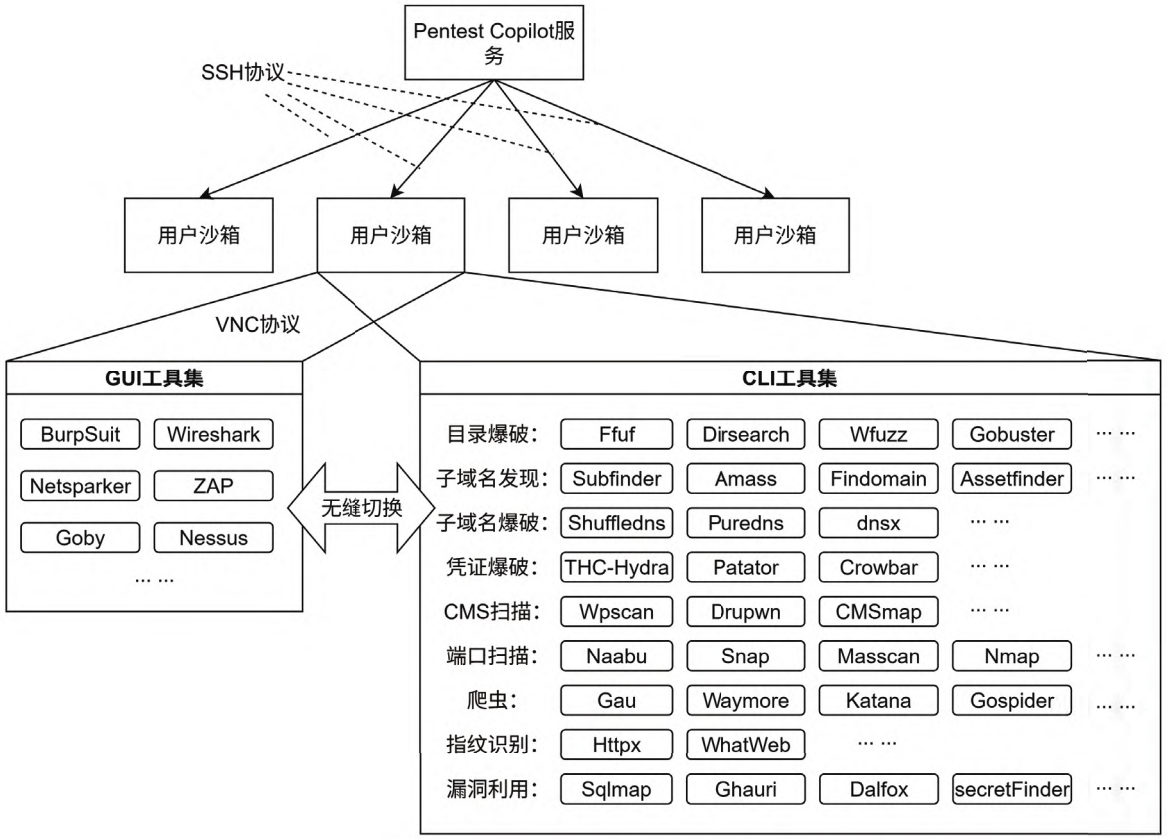


图 10 Pentest Copilot 工具集容器

对多种执行错误。首先，采用“3 次重试机制”，在每次执行后评估命令结果的有效性，若结果不符合预期，则自动优化参数并重新执行，3 次失败后将任务标记为失败并反馈给推理模块；其次，针对超时问题，为不同命令设置初始超时阈值，若超时发生，则优先选择效率更高的替代工具，若无替代工具，则动态调整阈值以适应网络延迟；最后，面对依赖缺失问题，框架采用“快速失败”策略，立即终止任务并将缺失依赖反馈给推理模块和人类操作员，防止资源浪费。

总体来看，这两类技术路径在不同层面提升了智能体的自我纠错和任务恢复能力，为构建高鲁棒性、低冗余的自动化渗透测试系统奠定了基础。

6 讨论

本章从 6 个维度系统分析了当前 LLMs 驱动的网络渗透测试智能体的主要不足，这些局限性相互关联，严重制约了框架自主化渗透测试的能力。针对性提出 6 项技术路径，围绕“感知—记忆—推理—执行—评估”的全流程构建改进闭环，提升智能体的多模态理解、长期经验复用、策略鲁棒性、创造性能力和可测控性，为渗透测试的智能化向真正自主化转型奠定坚实基础。

6.1 局限性及挑战

6.1.1 多模态识别和处理能力缺失

在渗透测试过程中，图像、视频等多模态信息以及图形化界面操作是判断漏洞存在性和利用路径的重要依据[55-56]。然而，现有框架普遍缺乏处理这些信息的能力，导致自动化流程中存在信息断层。例如，BreachSeek 尚未支持多模态输入，无法接收图像、视频等非文本数据，因此依赖视觉反馈的场景无法纳入自动化测试范畴，仍需人工补充信息；PentestGPT 在处理图像信息方面存在盲区，无法解析验证码或图形化配置界面中的漏洞信息，进而导致错失关键线索，限制了测试场景的覆盖；VulnBot 完全无法适配图形用户界面，必须由人工将图形化操作结果转化为文本描述后才能继续执行测试，这一人工干预环节成为其实现“端到端全自动化”的核心瓶颈，显著降

中国知网 https://www.cnki.net

低了测试的连续性和效率。

6.1.2 自动化程度与用户交互依赖的矛盾

自动化是网络渗透测试智能体框架的核心目标，但目前一些框架仍依赖人工交互来完成关键步骤，这导致了自动化效率和人工依赖之间的矛盾，影响了流程的连续性[57-58]。PentestAgent 在某些漏洞利用环节需要用户手动完成文件构造和上传，无法实现漏洞触发的完整自动化流程，从而打断了自动化链条，并增加了人工操作成本。VulnBot 除了需要人工描述图形界面外，在漏洞信息补全和工具执行结果解读等环节也需人工介入，未能实现端到端的自动化，无法满足大规模、高效率渗透测试的需求。PentestGPT 通过渗透测试任务树维护任务上下文，但在整个过程中仍然需要人工反馈和决策，导致测试时间和成本增加。

6.1.3 上下文感知与长时记忆管理缺陷

渗透测试是一个多步骤、强关联的动态过程，需要框架持续维护攻击目标、历史操作和当前策略的上下文，并根据长期记忆优化后续策略。若框架在这方面能力不足，会导致策略连贯性和容错性较差[59-60]。RedTeamLLM 没有维护关于智能体整体攻击目标的全局上下文，在多阶段渗透过程中容易遗漏关键信息，导致后续策略与最初目标脱节，进而出现攻击路径冗余或关键步骤缺失。AutoAttacker 的内存管理机制存在缺陷，只将内存用于存储“最新观察结果”，未能实现分层计划改进、长期记忆存储和故障计划回滚功能，导致在某一步骤执行失败时无法根据历史经验调整策略，只能重新启动或终止流程，容错性极低。PentestGPT 的计划和执行模块虽然能在推理、生成和解析阶段传递中间结果，但执行阶段启动后无法回访早期策略，导致当前路径受阻时无法切换到备选方案，策略灵活性严重不足。

6.1.4 漏洞检测利用能力的局限性

准确识别目标组件、探测潜在漏洞并生成有效的利用脚本是渗透测试的核心技术环节[61-62]。然而现有框架在这一环节的不足，将直接影响攻击渗透率。PentestAgent 无法识别目标系统中特定粒度的应用组件，且其依赖 Nmap 等传统枚举工具无法覆盖这些应用组件，导致无法有效匹配组件版本和已知漏洞，从而错失关键攻击入口；HPTSA 的 Agent 模块缺乏对非显式输入界面的探索能力，只能针对直观可见的输入接口进行漏洞探测，无法像人类测试员一样探测隐藏接口、配置文件和日志文件等“非直观攻击面”，导致漏洞探测的覆盖范围有限；HackingBuddyGPT[63] 在设置用户 ID（Set User ID，SUID）二进制文件检测中，无法像经验丰富的测试员那样判断“无害 SUID 文件”，仍需对所有 SUID 文件进行逐一检测，导致测试效率低下并产生大量无效告警；PentestGPT 在有限的尝试次数内难以生成细节完整的漏洞利用脚本，尤其是在涉及底层字节码操作时，生成的脚本常存在语法错误、参数不匹配或系统兼容性问题，无法直接用于漏洞触发。

6.1.5 经验迁移与创造性攻击思维缺失

在真实渗透测试中，人类测试员通常会基于历史案例进行经验迁移，并通过创造性思维探索非常规攻击路径[64-65]。框架在这方面的能力不足，制约了复杂场景下的渗透成功率。HackingBuddyGPT 的攻击策略依赖于规则枚举，缺乏对历史渗透案例的经验迁移能力，同时其策略生成仅限于预设模板，无法探索非常规的攻击路径，缺少创造性思维；Pentest Copilot 则存在攻击模式单一化的问题，其攻击行为偏向于使用常规的攻击模式，而这些模式已经被广泛应用，容易被目标系统的防护机制识别，从而导致攻击过程被阻断，降低了渗透测试的隐蔽性和成功率。

6.1.6 评估体系与现实场景的割裂问题

当前渗透测试智能体评估体系的有效性在很大程度上取决于评估场景与真实网络环境的匹配度[66]。要使评估结果对实际安全测试具有指导价值，评估框架必须能够模拟真实场景中的防护与攻击博弈、完整的攻击链闭环，以及多样化的任务需求[67]。表 4 展示了主流 LLMs 驱动的网络渗透测试智能体在模型选择与评估场景中的应用现状：常用靶场包括 HTB[68]、VulnHub[69]、MITRE ATT&CK、OWASP[70]、Metasploitable2、AUTOPENBENCH[71] 等，以及用于夺旗赛（Capture The Flag，CTF）场景评估的 NYU CTF Bench[72]、PicoCTF[73] 等。尽管这些评估框架已经根据场景难度进行区分，并覆盖了渗透测试的各流程，但大多数任务还是集中在常见漏洞上，难以充分匹配真实网络环境中复杂的防御态势。以 AutoAttacker 为例，其评估体系在环境预设、攻击覆盖和任务定位等方面存在显著不足：在环境预设方面，目标常被设定为漏洞裸露状态，忽视了真实环境中漏洞利用与防护机制间的动态博弈，导致漏洞利用成功率和攻击路径有效性等核

中国知网 https://www.cnki.net

表 4 主流 LLMs 驱动的网络渗透测试智能体框架模型使用及评估场景

框架名称	模型使用	评估场景
AutoAttacker	GPT-3.5, GPT-4, Llama 2-7B-chat, Llama 2-70B-chat	MITRE ATT&CK 中的 14 个任务
AutoPentest	GPT-4o	HTB（Devvortex，Broker，Codify）虚拟机
PenHeal	GPT-4 (128k tokens), GPT-3.5-turbo (16k tokens)	Metasploitable2
PentestAgent	GPT-3.5(gpt-3.5-turbo-0125), GPT-4(gpt-4o)	基于 VulnHub 开源漏洞 Docker 环境，涵盖不同难度和常见弱点，提供评估 PT 工具的实用框架
PentestGPT	PENTESTGPT-GPT-3.5, PENTESTGPT-GPT-4	包含来自 HackTheBox 和 VulnHub 的真实测试目标，共 13 个目标，182 个子任务，覆盖 OWASP Top 10 漏洞 和 18 个 CWE 项目
BreachSeek	Claude 3.5 Sonnet	Metasploitable2
VulnBot	GPT-4o，Llama3.3-70B, Llama3.1-405B, PentestGPT	AUTOPENBENCH 多种任务 AI-Pentest-Benchmark 13 台真实漏洞机器
AutoGPT	GPT-3.5，GPT-4o，GPT-4o mini	VulnHub 的 4 大类、6 个子类渗透测试目标
DJcipher	Claude 3.5 Sonnet, GPT-4o, GPT-4 Turbo, Llama 3.1 405B, Gemini 1.5 Flash	NYU CTF Bench，Cybench，HTB 共计 290 个 CTF 挑战，涵盖 6 个类别
HPTSA	GPT-4, Llama-3.1, Qwen-2.5	14 个真实世界的 Web 漏洞，涵盖 XSS、CSRF、SQLi、ACE 等
HackSynth	GPT-4o, GPT-4o-mini, Llama-3.1-8B, Llama-3.1-70B Qwen2-72B, Mixtral-8x7B, Phi-3-mini-4k, Phi-3.5-MoE	41 个 PicoCTF 挑战 32 个 OverTheWire 挑战
Pentest-Copilot	GPT-3.5-Turbo, GPT-4, GPT-4-Turbo, GPT-4o, GPT-4o-mini	“boot2root”服务器 30 个手动编排的测试用例
RefPentester	GPT-4o	HTB 平台的 Sau 机器
RedTeamLLMs	GPT-4o	VulnHub 的 5 个（Sar，CewiKid，Victim1，WestWild，CTF4）挑战

心指标均建立在理想化假设之上；在攻击覆盖方面，仅涵盖 MITRE ATT&CK 的 1 个小子集，缺乏对信息收集、初始访问和防御规避等关键战术的全面评估；在任务定位方面，评估通常局限于特定后渗透任务，未能构建适应多场景、多目标需求的评估模块，导致评估结果与实际渗透测试任务严重不符。为了提升评估体系的现实相关性，亟须将防御—攻击的博弈过程、更加全面的战术覆盖和多场景任务适配纳入评估设计中。

6.2 未来研究方向

6.2.1 多模态特征融合与图形界面交互技术

多模态特征融合与图形界面交互技术旨在解决现有 LLMs 驱动的网络渗透测试智能体无法处理图像、视频等非文本输入以及难以适应图形用户界面（Graphical User Interface，GUI）场景的问题，目标是实现“文本—图像—GUI 控件”的多模态信息闭环处理，从而消除人工干预瓶颈。其实现路径可以分为 3 个阶段。

1）构建多模态数据预处理与标注体系

基于对比语言—图像预训练模型[74]（Contrastive Language-Image Pre-training，CLIP）、ALBEF（Align Before Fuse）[75] 等“视觉—语言”预训练模型提取多模态特征，并结合渗透测试领域的知识进行权重优化。对于图像、视频以及相关文本数据，采用“人工标注 + 半自动化辅助”模式：利用 Label Studio[76] 标注 GUI 控件的坐标与功能，并借助 CLIP 的零样本分类生成初始标签，再通过人工修正提高标注精度和效率。

2）进行跨模态特征融合与语义对齐

中国知网 https://www.cnki.net

通过引入跨模态注意力机制[77]，在视觉编码器与文本编码器之间加入模态交互层，实现 GUI 控件的视觉特征与文本指令的语义动态关联。同时，采用场景自适应微调[78]，根据不同渗透测试场景调整特征权重，例如，在路由器管理界面中加强控件位置特征，在桌面软件场景中突出菜单路径和漏洞触发点的关联。

3）实现了 GUI 控件的自动化识别与操作执行通过结合计算机视觉和自动化工具链，构建了“控件识别—指令生成—执行验证”的端到端流程。首先，利用开放源代码计算机视觉库（Open source Computer Vision library,OpenCV）[79] 预训练的检测模型识别控件的类型和坐标；然后，通过 PyAutoGUI[80]、SikuliX[81]、Selenium[82] 等工具执行模型生成的操作指令；最后，通过截图对比或文本提取方式验证操作结果，形成反馈闭环。

6.2.2 分层记忆与强化学习融合技术

分层记忆与强化学习融合技术旨在解决现有 LLMs 驱动的网络渗透测试智能体在上下文丢失、长时记忆缺失及策略不可回滚等问题，构建 1 个“记忆→推理→执行→优化”闭环的自适应智能体。如图 11 所示，具体设计包括 3 个部分。

1）分层记忆[83]模块

借鉴人类记忆模型，构建包含临时缓存、长期存储和快速检索的 3 层体系。临时缓存用于存储当前渗透阶段的短期上下文，以保证低延迟访问，并在当前攻击路径结束时失效；长期存储通过向量数据库保存历史渗透经验、失败路径和成功案例；快速检索则使用余弦相似度等向量检索方法，实现任务需求与相关记忆的快速匹配。

2）强化学习驱动的策略迭代与故障回滚

将渗透过程建模为马尔可夫决策过程[84]

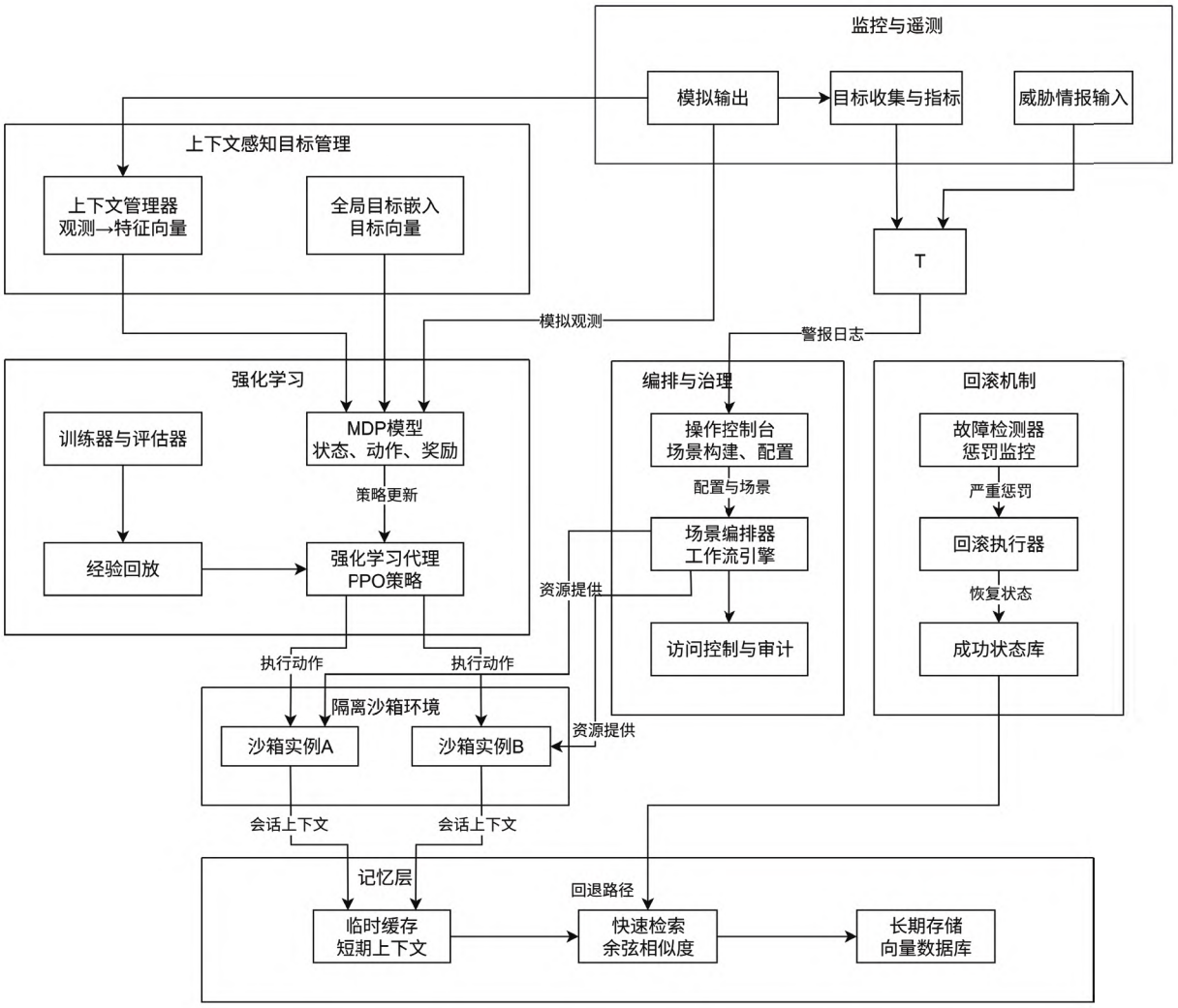


图 11　分层记忆与强化学习融合技术架构图

中国知网　https://www.cnki.net

（Markov Decision Process，MDP），定义状态与动作，并设计考虑正负向信号的奖励函数。使用近端策略优化[85]（Proximal Policy Optimization，PPO）等策略梯度算法结合经验回放进行策略收敛与迁移。当某一动作触发严重惩罚时，系统会自动执行故障回滚，从工作记忆中检索最近的成功状态，回滚至该状态并从长期记忆中调取备选攻击路径，保证流程的连续性与鲁棒性。

最后是上下文感知的策略连贯性保障。引入全局目标嵌入[86]机制，将总体攻击目标编码为向量，并将其融入每一步决策过程中；计算全局目标向量与当前观测向量的相似度，实现优先级调整与策略聚焦，避免关键信息遗漏，确保每一步操作服务于整体目标。

6.2.3 知识图谱驱动的精细化漏洞检测技术

为解决当前LLMs驱动的网络渗透测试智能体在组件识别精度低、漏洞探测逻辑单一等方面的局限，可探索知识图谱驱动的精细化漏洞检测技术，旨在实现“组件—漏洞—攻击路径”的精准匹配，并主动发现非直观漏洞，包含4个关键步骤。

1）构建渗透测试专用的知识图谱[87]

通过本体论定义核心实体和关系，利用结构化与非结构化数据进行知识抽取，并使用图数据库进行存储和定期更新。知识图谱为后续的推理与匹配提供规范化的语义支持和时效保障，从组件到漏洞及攻击技术的语义映射得以实现。

2）多源信息融合以实现细粒度的组件识别[88]

结合端口扫描、进程分析、文件指纹和日志提取等多层次特征，构建跨层次的证据链：端口层提供服务头和证书信息，进程层获取运行命令和参数，文件层读取版本信息，日志层提取请求特征。通过将这些特征输入知识图谱进行关联推理，可提升对轻量级或非标准组件的识别精度和版本判定能力。

3）采用主动探索算法探测非直观漏洞

将目标系统抽象为状态树，结合蒙特卡洛树搜索（Monte Carlo tree search，MCTS）和经验迁移的主动探索策略，按照选择、扩展、模拟、回溯的步骤进行操作[89]。同时，利用历史经验指导MCTS的初始化方向，减少盲目探索并提高非直观漏洞的发现效率。

4）建立经验驱动的无效探测过滤机制

通过迁移学习与规则库构建“无害组件和操作”的过滤机制，减少噪声和无效尝试。离线总结历史经验中的无害特征并形成规则库，在线执行时进行匹配并跳过无害的探测目标；同时，通过强化学习的负向反馈周期性更新规则，提升过滤的准确性与自适应性。

6.2.4 对抗性训练与创造性生成技术

为解决现有LLMs驱动的网络渗透测试智能体在创造性思维不足和攻击模式单一等方面的局限，可探索对抗性训练与创造性生成技术，这是1种结合案例驱动的生成式人工智能（Generative Artificial Intelligence，GenAI）与对抗训练的框架[90]，旨在实现经验迁移、创造性路径合成和攻击隐蔽性的增强。该框架由3个部分组成：结构化案例库构建、创造性攻击路径生成和对抗训练以提升隐蔽性。

1）结构化案例库构建

渗透测试案例知识库的结构化构建，通过收集公开的渗透测试报告、红蓝对抗日志和漏洞复现记录，按照目标环境、攻击链、关键步骤和防御规避手段进行结构化存储。采用事件抽取和信息抽取模型提取并归一化关键要素，以便后续进行检索、相似性匹配和向量化嵌入。

2）创造性攻击路径生成[91]

一是将结构化案例进行向量化嵌入，形成案例表示；二是面对新目标时，通过计算目标向量与案例向量的相似度，召回相关案例；三是利用LLMs的生成能力，对召回的攻击链在语义和操作层面进行改造与扩展，生成适配新环境的创造性路径。该过程可结合约束条件和校验步骤，确保生成的路径既实用又安全可控。

3）对抗训练以提升攻击隐蔽性

通过构建“渗透测试—防御检测”的对抗训练闭环，将入侵检测系统[92]（Intrusion Detection System，IDS）、端点检测和响应[93]（Endpoint Detection and Response，EDR）等检测器的规则与行为特征纳入训练反馈。通过生成对抗样本，并以检测器的响应作为负向奖励，优化生成模型，使其在保持攻击有效性的同时降低被检测的概率。对抗训练可以采用强化学习或对抗生成网络[94]（Generative Adversarial Network，GAN）范式，并通过离线仿真和受控评估验证隐蔽性改进效果。

6.2.5 工具认知对齐与协议增强技术

工具认知对齐[95]与协议增强技术，旨在解决

中国知网 https://www.cnki.net

当前 LLMs 驱动的网络渗透测试智能体在工具调用方面的认知偏差、协议适配不足和闭环执行能力薄弱的问题。该架构提出“工具认知对齐—协议动态适配—闭环迭代优化”的三位一体技术框架，旨在实现对工具功能、参数逻辑和环境依赖的深度理解，并支撑从“意图解析—工具选型—参数生成—执行反馈—策略调整”的端到端自动化调用流程。主要工作可以分为 2 个模块和配套的安全保障机制。

1）渗透测试工具认知对齐模块 [96]

该模块可构建一个包含工具元数据、参数逻辑、场景适配和错误案例的多维知识图谱，结合 Neo4j[97] 存储和 RetrievalQA[98] 机制实现知识增强，帮助 LLMs 在上下文感知方面做出正确的工具选择与参数约束；此外，通过基于容器化高保真漏洞环境自动生成训练样本，采用 LoRA[99]（Low-Rank Adaptation）微调技术，帮助模型学习工具选择的合理性和输出解析的漏洞语义关联。同时，利用双向编码器表征模型 [100]（Bidirectional Encoder Representations from Transformers，BERT）和中文命名实体识别（Named Entity Recognition，NER）技术，训练意图识别模型，识别信息收集、漏洞探测、权限提升、横向移动和数据窃取 5 类核心意图，并结合环境探针模块实时获取目标上下文，实现工具调用的动态适配与参数自调整 [101]。

2）场景增强型模块

场景增强上下文协议 [102]（Model Context Protocol，MCP）如图 12 所示，将传统 MCP 的 messages、state、tools 字段扩展为包含“pen_test_ext”的 6 维结构，引入上下文状态机、工具调用规范与反馈解析规则，实现协议与渗透流程的语义耦合。采用 ReAct 风格的“意图解析—协议封装—工具执行—结果解析—状态更新”闭环交互机制，并嵌入工具链优先级矩阵和冲突解决策略，由 Orchestrator 基于“target_context”和“pen_test_ext”进行跨工具动态调度和冲突化解。

3）安全性与可靠性保障机制

通过在 MCP Server 层预置了高危操作黑名单和风险评估机制，对模型生成的调用指令进行前置审查和分级控制。对于高风险操作，需要人工确认后才能执行，并配备日志记录、可审计和状态回滚机制，以防止异常操作并提高系统的鲁棒性 [103]。

6.2.6 建立逼真且可比对的评估与基准体系

为了缩小实验环境与真实场景之间的差距，并在前、中、后序战术层面真实反映攻防博弈，构建 1 套具备可比较性与重现性的评估与基准体

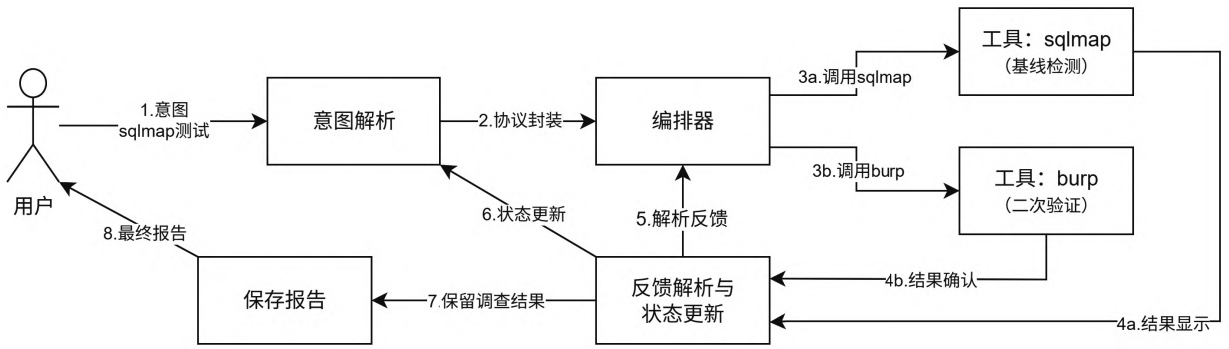


图 12　场景化增强型 MCP 技术流程图

系，包括 4 个互补的路径。

首先，构建多维评估矩阵，采用覆盖率、隐蔽性、操作成功率、误报率、恢复能力和资源消耗等相关指标，形成 1 套可量化的复合度量体系，从而全面、可比较地评估智能体在不同作业目标和约束下的综合性能。

其次，搭建拟真实环境的仿真平台 [104]，集成主动防御组件和可配置的蓝队策略，使得“渗透—防御”博弈能够在不同防护强度和策略组合下开展可重复地演练与评估，为现实级测试场景提供可信的映射和对照。

再次，发布标准化与去敏感化处理的基准集与数据集 [105]，提供明确且公开的测试用例与对抗场景，包含任务说明、评价协议和元数据，使跨团队、跨机构的比较与复验成为可能，同时严格遵守隐私与合规约束。

中国知网　https://www.cnki.net

最后，定期开展红蓝对抗评测，在包含真实防护组件和监测能力的环境中，让红队智能体执行既定任务，由蓝队基于预定义的指标评估检测、响应和阻断能力，从攻防双方的角度检验智能体的实用性和局限性[106]。

7 结 语

本文系统地分析了LLMs驱动的网络渗透测试智能体的关键技术，构建了1套涵盖角色设定、任务规划、记忆管理和交互执行的模块化体系，展示了在提升效率和智能决策方面的核心突破。尽管现有智能体已具备一定的自主化能力，但在多模态信息处理、长时记忆保持、策略连贯性和组件识别精度等方面仍存在瓶颈，整体仍处于“半自主化”阶段。针对这些问题，本文结合当前智能体技术研究方向，提出了多模态信息融合、记忆增强与强化学习结合、知识图谱驱动的漏洞探测、生成式AI与对抗训练、增强型MCP工具调用架构以及标准化评估体系6条可探索的技术路径，为智能体向全自主化的演进提供了参考。

总体来看，本文验证了LLMs驱动的网络渗透测试智能体在网络安全测试智能化中的核心价值，并为未来的研究提供了技术框架和发展指引。未来，这一技术将推动渗透测试从“人工主导+智能辅助”向“智能主导+人工监督”转变，成为主动安全体系智能化升级的重要支撑。

参 考 文 献

[1] Du S, Zhao J, Shi J, et al. A survey on the optimization of large language model-based agents [J]. ACM Computing Surveys, 2026, 58(9): 1-37.

[2] 舒展，李宗鹏. 基于大语言模型的自动化渗透测试研究 [J]. 计算，2025，1（2）：23-30.

[3] Al-Shareefi F, Chu G, Lisitsa A. A multi-agent framework for penetration testing: Modelling and analysing using abstract state machines [C]//Proceedings of the 2024 IExEE International Symposium on Parallel and Distributed Processing with Applications. Piscataway: IEEE, 2024: 1021-1028.

[4] Nieponice T, Valeros V, Garcia S. ARACNE: An LLM-based autonomous shell pentesting agent[EB/OL]. 2025 [2026-06-22]. https://arxiv.org/abs/2502.18528.

[5] 占力戈，沙乐天，肖甫，等. 基于强化学习的自动化Windows域渗透方法 [J]. 网络与信息安全学报，2023，9（4）：104-120.

[6] Wang Y, Pan Y, Su Z, et al. Large Model-Based Agents: State-of-the-Art, Cooperation Paradigms, Security and Privacy, and Future Trends[EB/OL]. 2026 [2026-06-22]. https://ieeexplore.ieee.org/document/11022699/references#references.

[7] Han Y, Liu C, Wang P. A comprehensive survey on vector database: Storage and retrieval techniques, challenges[EB/OL]. 2023[2026-06-22]. https://arxiv.org/abs/2310.11703.

[8] 高康，汤博文，金建栋，等. 基于检索增强生成的智能渗透测试方法研究 [J]. 信息安全研究，2024，10（E2）：32.

[9] Zou Y, Hong Y, Xu J, et al. Leveraging large language models for challenge solving in capture-the-flag [C]//Proceedings of the 2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications. Piscataway: IEEE, 2024: 1541-1550.

[10] Hilario E, Azam S, Sundaram J, et al. Generative AI for pentesting: The good, the bad, the ugly [J]. International Journal of Information Security, 2024, 23(3): 2075-2097.

[11] Niu Z, Ren H, Gao X, et al. Jailbreaking attack against multimodal large language model[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2402.02309.

[12] Ma J, Li Y, Xiao Z, et al. Jailbreaking prompt attack: A controllable adversarial attack against diffusion models[C]//Findings of the Association for Computational Linguistics: NAACL 2025. Albuquerque: Association for Computational Linguistics, 2025: 3141-3157.

[13] Xu J, Stokes J W, McDonald G, et al. Autoattacker: A large language model guided system to implement automatic cyber-attacks[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2403.01038.

[14] Nmap Project. Nmap: The network mapper[EB/OL]. 2025[2026-06-22]. https://nmap.org/.

[15] Shen X, Wang L, Li Z, et al. Pentestagent: Incorporating LLM agents to automated penetration testing[C]//Proceedings of the 20th ACM Asia Conference on Computer and Communications Security. New York: ACM, 2025: 375-391.

[16] Wang K, Stolfo S J. Anomalous payload-based network intrusion detection[C]//Proceedings of the International Workshop on Recent Advances in Intrusion Detection. Berlin Heidelberg: Springer, 2004: 203-222.

[17] Zhu Y, Kellermann A, Gupta A, et al. Teams of LLM agents can exploit zero-day vulnerabilities[C]//Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics, Volume 1: Long Papers. Rabat: Association for Computational Linguistics, 2026: 23-35.

[18] Érdi A. Simulating web user behavior using LLM-driven browser

中国知网 https://www.cnki.net

automation for realistic IDS dataset generation[D]. Amsterdam: University of Applied Sciences, 2025.

[19] Deng G, Liu Y, Mayoral-Vilches V, et al. PentestGPT: Evaluating and harnessing large language models for automated penetration testing[C]//Proceedings of the 33rd USENIX Security Symposium. Berkeley: USENIX Association, 2024: 847-864.

[20] Dubois P. MySQL[M]. Upper Saddle River: Addison-Wesley, 2013.

[21] Alshehri I, Alshehri A, Almalki A, et al. BreachSeek: A multi-agent automated penetration tester[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2409.03789.

[22] Pelluru K. LangChain & LangGraph in production: Architectures for multi-agent LLM systems[J]. Journal of Data and Digital Innovation, 2025, 2(3): 1-9.

[23] Guo T, Chen X, Wang Y, et al. Large language model based multi-agents: A survey of progress and challenges[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2402.01680.

[24] Jiang T. AutoPenGPT: Highly automated penetration testing framework based on LLM[D]. Auckland: University of Auckland, 2025.

[25] Shi C, Yang C, Zhu X, et al. Unchosen experts can contribute too: Unleashing MoE models' power by self-contrast[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2405.14507.

[26] Kong H, Hu D, Ge J, et al. Vulnbot: Autonomous penetration testing for a multi-agent collaborative framework[EB/OL]. 2025[2026-06-22]. https://arxiv.org/abs/2501.13411.

[27] Wu B, Chen G, Chen K, et al. AutoPT: How far are we from the end-to-end automated web penetration testing?[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2411.01236.

[28] Cabrera E F, de Barros M R, Costa A H R. Improving LLMs' reasoning and planning with finite-state machines[C]//Proceedings of the Brazilian Conference on Intelligent Systems. Cham: Springer Nature Switzerland, 2024: 110-124.

[29] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing reasoning and acting in language models[EB/OL]. 2023[2026-06-22]. https://openreview.net/forum?id=WE_vluYUL-X.

[30] Liu Z, Bai X, Chen K, et al. A survey on the feedback mechanism of LLM-based AI agents[C]//Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence. Montreal: IJCAI, 2025: 10582-10592.

[31] Gao Y, Xiong Y, Gao X, et al. Retrieval-augmented generation for large language models: A survey[EB/OL]. 2023[2026-06-22]. https://arxiv.org/abs/2312.10997.

[32] Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks[EB/OL]. 2020[2026-06-22]. https://arxiv.org/abs/2005.11401v4.

[33] Huang J, Zhu Q. Penheal: A two-stage LLM framework for automated pentesting and optimal remediation[C]//Proceedings of the Workshop on Autonomous Cybersecurity. Salt Lake City: ACM, 2023: 11-22.

[34] He X, Yang D, Feng W, et al. CPL: Counterfactual prompt learning for vision and language models[EB/OL]. 2022[2026-06-22]. https://arxiv.org/abs/2210.10362.

[35] Fontana N. LLM's dilemma: Analyzing the behaviors of large language models in the iterated prisoner's dilemma[D]. Milan: Politecnico di Milano, 2023.

[36] Dai H, Li Y, Yan J, et al. RefPentester: A knowledge-informed self-reflective penetration testing framework based on large language models[EB/OL]. 2025[2026-06-22]. https://arxiv.org/abs/2505.07089.

[37] 周仕承，刘京菊，钟晓峰，等 . 基于深度强化学习的智能化渗透测试路径发现 [J]. 计算机科学，2021，48（7）：40-46.

[38] Challita B, Parrend P. RedTeamLLM: An agentic AI framework for offensive security [EB/OL]. 2025[2026-06-22]. https://arxiv.org/abs/2505.06913.

[39] Lu Z, Lu W, Tao Y, et al. Decompose, plan in parallel, and merge: A novel paradigm for large language models based planning with multiple constraints[EB/OL]. 2025[2026-06-22]. https://arxiv.org/abs/2506.02683.

[40] Goyal D, Subramanian S, Peela A, et al. Hacking, the lazy way: LLM augmented pentesting[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2409.09493.

[41] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models[EB/OL]. 2022[2026-06-22]. https://arxiv.org/abs/2201.11903.

[42] Pratama D, Suryanto N, Adiputra A A, et al. Cipher: Cybersecurity intelligent penetration-testing helper for ethical researchers[J]. Sensors, 2024, 24(21): 6878.

[43] Sun J, Xing Z, Guo H, et al. Generating informative CVE description from ExploitDB posts by extractive summarization[EB/OL]. 2021[2026-06-22]. https://arxiv.org/abs/2101.01431.

[44] Fan W, Ding Y, Ning L, et al. A survey on RAG meeting LLMs: Towards retrieval-augmented large language models[C]//Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. New York: ACM, 2024: 6491-6501.

[45] Mitre Corporation. MITRE ATT&CK[EB/OL]. 2025[2026-06-22]. https://attack.mitre.org/.

[46] Bach-Nutman M. Understanding the top 10 OWASP vulnerabilities[EB/OL]. 2020[2026-06-22]. https://arxiv.org/

中国知网 https://www.cnki.net

abs/2012.09960.

[47] Henke J. AutoPentest: Enhancing vulnerability management with autonomous LLM agents[EB/OL]. 2025[2026-06-22]. https://arxiv.org/abs/2505.10321.

[48] Tavily-AI. Tavily MCP: Production ready MCP server with real-time search, extract, map and crawl[EB/OL]. 2025[2026-06-22]. https://github.com/tavily-ai/tavily-mcp.

[49] PortSwigger. Burp Suite Community Edition[EB/OL]. [2026-06-22]. https://portswigger.net/burp/communitydownload.

[50] Guimaraes B, Stampar M. sqlmap: Automatic SQL injection and database takeover tool[EB/OL]. 2006[2026-06-22]. https://sqlmap.org/.

[51] Rapid7. Metasploit framework[EB/OL]. 2003[2026-06-22]. https://www.metasploit.com/.

[52] Wireshark Foundation. Wireshark: Network protocol analyzer[EB/OL]. 2006[2026-06-22]. https://www.wireshark.org/.

[53] Ginige Y, Niroshan A, Jain S, et al. AutoPentester: An LLM agent-based framework for automated pentesting[C]//2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications. Piscataway: IEEE, 2025: 163-174.

[54] Nakatani S. RapidPen: Fully automated IP-to-shell penetration testing with LLM-based agents[EB/OL]. 2025[2026-06-22]. https://arxiv.org/pdf/2502.16730.

[55] Fei H, Yao Y, Zhang Z, et al. From multimodal LLM to human-level AI: Modality, instruction, reasoning, efficiency and beyond[C]//Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation: Tutorial Summaries. Torino: ELRA and ICCL, 2024: 1-8.

[56] Wang Y, Wang L, Zhou Q, et al. Multimodal LLM enhanced cross-lingual cross-modal retrieval[C]//Proceedings of the 32nd ACM International Conference on Multimedia. New York: ACM, 2024: 8296-8305.

[57] Ferrag M A, Tihanyi N, Debbah M. From LLM reasoning to autonomous AI agents: A comprehensive review[EB/OL]. 2025[2026-06-22]. https://arxiv.org/abs/2504.19678.

[58] Ifargan T, Hafner L, Kern M, et al. Autonomous LLM-driven research: From data to human-verifiable research papers[J]. NEJM AI, 2025, 2(1): AIoa2400555.

[59] An S, Ma Z, Lin Z, et al. Make your LLM fully utilize the context[EB/OL]. 2024[2026-06-22]. https://papers.neurips.cc/paper_files/paper/2024/hash/71c3451f6cd6a4f82bb822db25cea4fd-Abstract-Conference.html.

[60] Jin H, Han X, Yang J, et al. LLM maybe LongLM: Self-extend LLM context window without tuning[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2401.01325.

[61] Lu G, Ju X, Chen X, et al. GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning[EB/OL]. 2024[2026-06-22]. https://dl.acm.org/doi/abs/10.1016/j.jss.2024.112031.

[62] Akuthota V, Kasula R, Sumona S T, et al. Vulnerability detection and monitoring using LLM[C]//2023 IEEE 9th International Women in Engineering Conference on Electrical and Computer Engineering. Piscataway: IEEE, 2023: 309-314.

[63] Happe A, Kaplan A, Cito J. LLMs as hackers: Autonomous Linux privilege escalation attacks[EB/OL]. 2023[2026-06-22]. https://ui.adsabs.harvard.edu/abs/2023arXiv231011409H/abstract.

[64] Lu L C, Chen S J, Pai T M, et al. LLM discussion: Enhancing the creativity of large language models via discussion framework and role-play[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2405.06373.

[65] Zhao Y, Zhang R, Li W, et al. Assessing and understanding creativity in large language models[J]. Machine Intelligence Research, 2025, 22(3): 417-436.

[66] Li H, Dong Q, Chen J, et al. LLMs-as-judges: A comprehensive survey on LLM-based evaluation methods[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2412.05579.

[67] Isozaki I, Shrestha M, Console R, et al. Towards automated penetration testing: Introducing LLM benchmark, analysis, and improvements[C]//Adjunct Proceedings of the 33rd ACM Conference on User Modeling, Adaptation and Personalization. New York: ACM, 2025: 404-419.

[68] Hack The Box. Hack The Box: Cybersecurity performance center[EB/OL]. 2024[2026-06-22]. https://www.hackthebox.com/.

[69] VulnHub. VulnHub: Virtual machines for penetration testing and ethical hacking[EB/OL]. 2012[2026-06-22]. https://www.vulnhub.com/.

[70] OWASP Foundation. OWASP Juice Shop Project[EB/OL]. 2025[2026-06-22]. https://owasp.org/www-project-juice-shop/.

[71] Gioacchini L, Mellia M, Drago I, et al. Autopenbench: Benchmarking generative agents for penetration testing[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2410.03225.

[72] Shao M, Jancheska S, Udeshi M, et al. NYU CTF Bench: A scalable open-source benchmark dataset for evaluating LLMs in offensive security[EB/OL]. 2024[2026-06-22]. https://ar5iv.labs.arxiv.org/html/2406.05590.

[73] picoCTF. picoCTF competitions: 2021 redpwn[EB/OL]. 2025[2026-06-22]. https://picoctf.org/competitions/2021-redpwn.html.

[74] Huang W, Wu A, Yang Y, et al. LLM2CLIP: Powerful language model

中国知网 https://www.cnki.net

unlocks richer visual representation[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2411.04997.

[75] Song C. Enhancing multimodal understanding with LIUS[J]. Journal of Organizational and End User Computing, 2024, 36(1): 1-17.

[76] Faisal S, Mehmood Z, Rafay M A, et al. Adaptive multi modal annotation for high quality, scalable machine learning data pipelines[J]. Spectrum of Engineering Sciences, 2025, 3(7): 150-160.

[77] Kumar A, Naseer M, Narayan S, et al. Multi-modal generation via cross-modal in-context learning[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2405.18304.

[78] Han J, Zhang R, Shao W, et al. ImageBind-LLM: Multi-modality instruction tuning[EB/OL]. 2023[2026-06-22]. https://arxiv.org/abs/2309.03905.

[79] Bradski G. The OpenCV Library[J]. Dr. Dobb's Journal of Software Tools, 2000, 25(11):120-126.

[80] Sweigart A. PyAutoGUI documentation[EB/OL]. [2026-06-22]. https://pyautogui.readthedocs.io/.

[81] RaiMan. SikuliX version 2.0.0+[EB/OL]. 2025[2026-06-22]. https://github.com/RaiMan/SikuliX1.

[82] SeleniumHQ. Selenium: A browser automation framework and ecosystem[EB/OL]. 2025[2026-06-22]. https://github.com/SeleniumHQ/selenium.

[83] Sun H, Zeng S. Hierarchical memory for high-efficiency long-term reasoning in LLM agents[EB/OL]. 2025[2026-06-22]. https://arxiv.org/abs/2507.22925.

[84] Hu B, Zhao C, Zhang P, et al. Enabling intelligent interactions between an agent and an LLM: A reinforcement learning approach[EB/OL]. 2023[2026-06-22]. https://arxiv.org/abs/2306.03604.

[85] Wang Z, Bi B, Pentyala S K, et al. A comprehensive survey of LLM alignment techniques: RLHF, RLAIF, PPO, DPO and more[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2407.16216.

[86] Chen J, Li H, Yang J, et al. Enhancing LLM-based agents via global planning and hierarchical execution[EB/OL]. 2025[2026-06-22]. https://arxiv.org/abs/2504.16563.

[87] Kommineni V K, König-Ries B, Samuel S. From human experts to machines: An LLM supported approach to ontology and knowledge graph construction[EB/OL]. 2024[2026-06-22]. https://ieeexplore.ieee.org/abstract/document/10822222/.

[88] Liu R, Xie Y, Dang Z, et al. Dynamic vulnerability knowledge graph construction via multi-source data fusion and large language model reasoning[J]. Electronics, 2025, 14(12): 2334.

[89] Zhang D, Zhoubian S, Hu Z, et al. ReST-MCTS*: LLM self-training via process reward guided tree search[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2406.03816.

[90] Zhu Q. Game theory meets LLM and agentic AI: Reimagining cybersecurity for the age of intelligent threats[EB/OL]. 2025[2026-06-22]. https://arxiv.org/abs/2507.10621.

[91] Fang R, Bindu R, Gupta A, et al. LLM agents can autonomously exploit one-day vulnerabilities[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2404.08144.

[92] Ashoor A S, Gore S. Importance of intrusion detection system[J]. International Journal of Scientific and Engineering Research, 2011, 2(1): 1-4.

[93] Kaur H, Sl D S, Paul T, et al. Evolution of endpoint detection and response in cyber security: A comprehensive review[C]//E3S Web of Conferences. Les Ulis: EDP Sciences, 2024, 556: 01006.

[94] Devadiga D, Jin G, Potdar B, et al. GLEAM: GAN and LLM for evasive adversarial malware[C]//2023 14th International Conference on Information and Communication Technology Convergence. Piscataway: IEEE, 2023: 53-58.

[95] Liu Y, Yao Y, Ton J F, et al. Trustworthy LLMs: A survey and guideline for evaluating large language models' alignment[EB/OL]. 2023[2026-06-22]. https://arxiv.org/abs/2308.05374.

[96] Liguori P, Marescalco C, Natella R, et al. The power of words: Generating PowerShell attacks from natural language[C]//18th USENIX WOOT Conference on Offensive Technologies. Berkeley: USENIX Association, 2024: 27-43.

[97] Vukotic A, Watt N, Abedrabbo T, et al. Neo4j in action[M]. Shelter Island: Manning Publications Co., 2014.

[98] Zhang Z, Fang M, Chen L. RetrievalQA: Assessing adaptive retrieval-augmented generation for short-form open-domain question answering[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2402.16457.

[99] Hasanov I, Virtanen S, Hakkala A, et al. Application of large language models in cybersecurity: A systematic literature review[EB/OL]. 2024[2026-06-22]. https://arxiv.org/abs/2405.04760.

[100] Kasri W, Himeur Y, Alkhazaleh H A, et al. From vulnerability to defense: The role of large language models in enhancing cybersecurity[J]. Computation, 2025, 13(2): 30.

[101] Haurogné J, Basheer N, Islam S. Vulnerability detection using BERT based LLM model with transparency obligation practice towards trustworthy AI[EB/OL].[C]//2025 IEEE International Conference on IT in Egypt (ITC-Egypt). Piscataway: IEEE, 2025.

[102] Ray P P. A survey on model context protocol: Architecture, state-of-the-art, challenges and future directions[EB/OL]. 2025[2026-06-22]. https://www.techrxiv.org/doi/full/10.36227/

中国知网 https://www.cnki.net

techrxiv.174495492. 22752319

[103] Hou X, Zhao Y, Wang S, et al. Model context protocol: Landscape, security threats, and future research directions[EB/OL]. 2025[2026-06-22]. https://dl.acm.org/doi/10.1145/3796519.

[104] Nguyen H P T, Chen Z, Hasegawa K, et al. PenGym: Pentesting training framework for reinforcement learning agents[C]// Proceedings of the 10th International Conference on Information Systems Security and Privacy - Volume 1: ICISSP. Rome: SciTePress, 2024: 498-509.

[105] Tihanyi N, Ferrag M A, Jain R, et al. CyberMetric: A benchmark dataset based on retrieval-augmented generation for evaluating LLMs in cybersecurity knowledge[C]//2024 IEEE International Conference on Cyber Security and Resilience. Piscataway: IEEE, 2024: 296-302.

[106] Soulé J, Jamont J P, Occello M, et al. Towards a multi-agent simulation of cyber-attackers and cyber-defenders battles[C]//2023 IEEE International Conference on Systems, Man, and Cybernetics. Piscataway: IEEE, 2023: 3594-3599.

作 者 简 介

申　晴（1989—），山西吕梁，硕士，工程师，主要研究方向为智能网络对抗、渗透测试。
sq_1223@sina.com

宋　磊（1976—），江苏南京，博士，副教授，主要研究方向为智能网络对抗、网络安全。
32299321@qq.com

周振吉（1985—），江苏南京，博士，副教授，主要研究方向为智能网络对抗、漏洞挖掘。
zhouzj@aeu.edu.cn

侯瑞博（1983—），山西太原，硕士，工程师，主要研究方向为网络安全、信息运维。
hrbww@sina.com

战权海（1997—），辽宁抚顺，博士，工程师，主要研究方向为智能网络对抗、密码学。
zhanquanhai@163.com

王拓夫（1992—），江苏南京，硕士，工程师，主要研究方向为网络对抗。
4937101@qq.com

中国知网 https://www.cnki.net